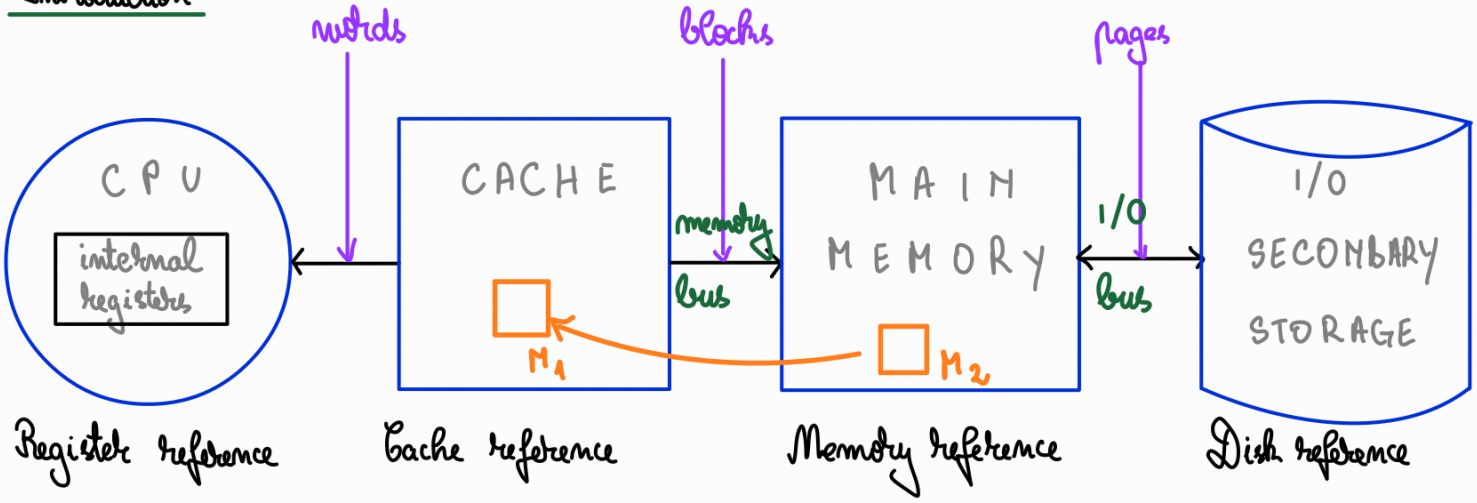
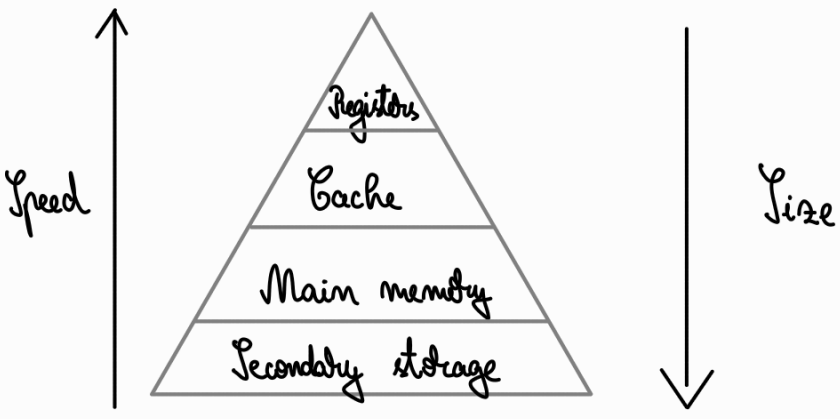
Chapter 3: Memory hierarchies (Ierarhia de memorie)

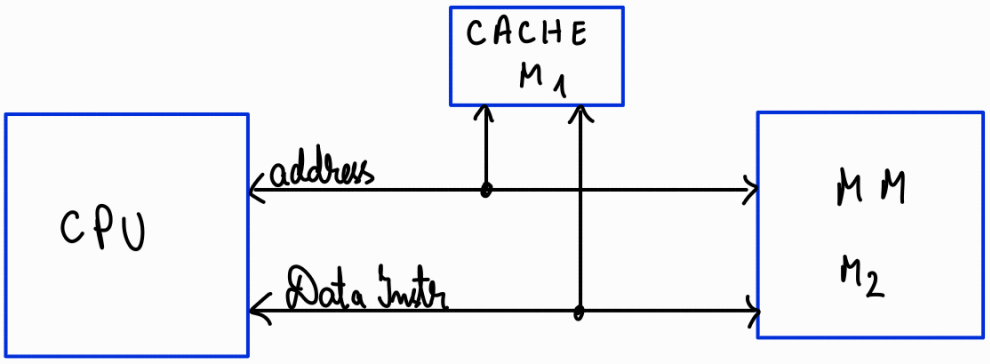
Introduction



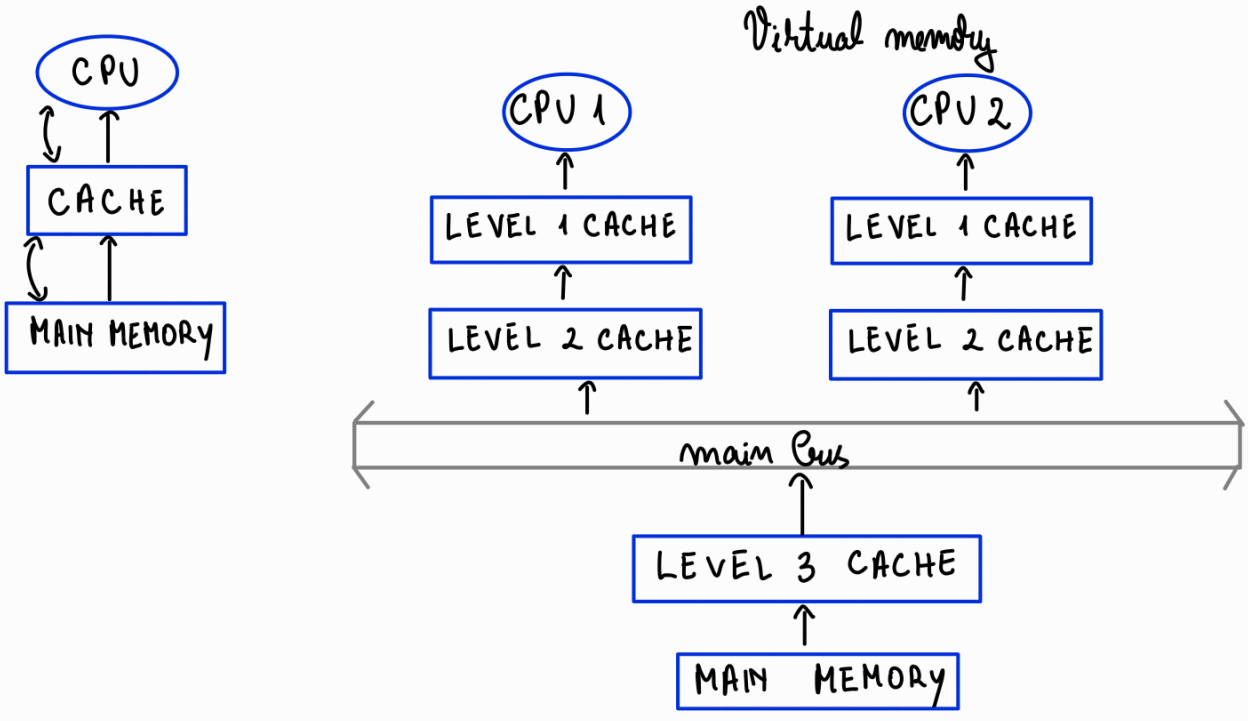


Locality principle

- temporal locality: când un program modifică un contor într-un loop, accesează aceași variabilă. Cache-ul păstrează datele folosite într-o memorie mai rapidă, pt. a nu accesa MM.
- spatial locality: la parcurgerea unui tablou, adresele de memorie sunt continue. E eficient să aduci în cache un întreg bloc de memorie (o linie de cache de 64 octeți poate conține A[0]...A[15] și după ce ai încărcat A[0], accesul la A[1] etc rămâne în cache)



- Cache e $\approx$ 10% din Main Memory
- M_1, M_2 — notații conform Patterson și Hennessy

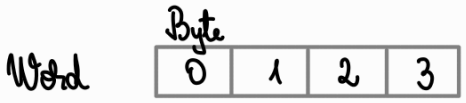


Taxonomy & basic notations

- Addressable unit → 1B = 1 Byte (generally)
  - 1 word = 2^r bytes , $r \in \mathbb{N}$
  - 1 Block = 2^n words
  - 1 cache hit / miss → miss rate

exe: $r = 2$ → 1 word = 2^2 bytes => word offset (W.O.) :

0 0
0 1
1 0
1 1



3.1. Caches

Cache coherency : cache
- unified
- split
  - → Data
  - → Instruction

3.1.1. Direct mapping (DM)

M_2: 2^m blocks , i - index , $i \in \{0, \dots, 2^m - 1\}$

M_1 : 2^n blocks , j - index , $j \in \{0, \dots, 2^n - 1\}$

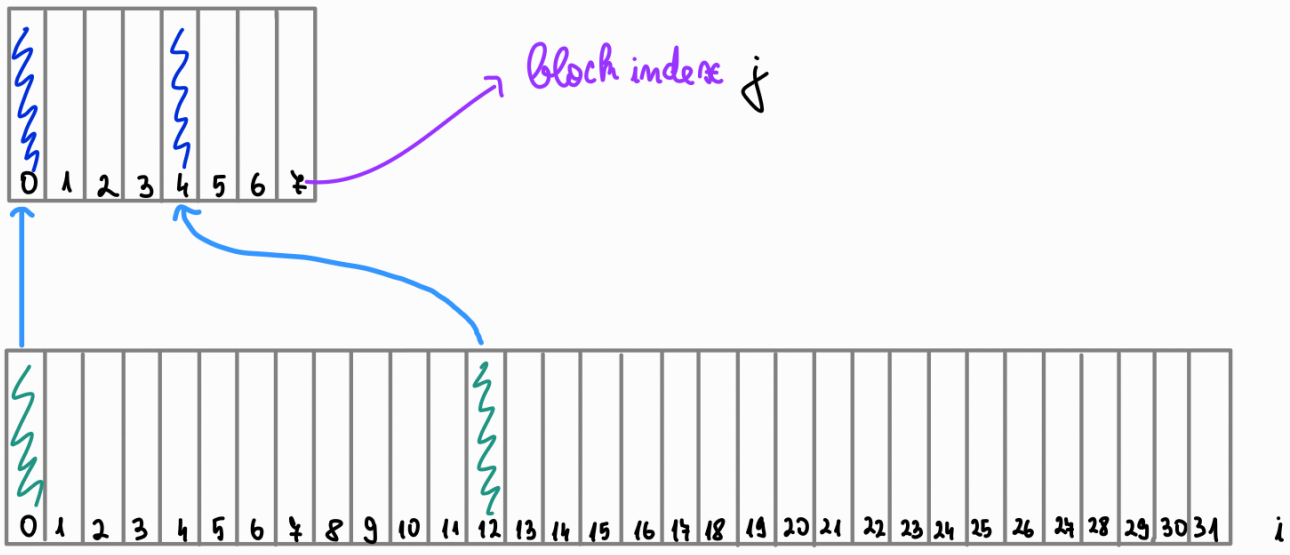
cu $m < n$

$j = i \bmod 2^m$ → locul unde se mapează $M_2(i)$

Ex: $n = 5$

$m = 3$

M_1 (2^3)



$M_2(2^5)$

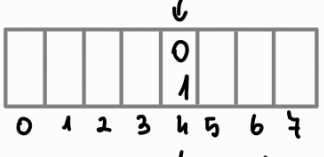
$i \to M_2$ $j \to M_1$

$j = i \bmod 2^m = 0 \bmod 2^3 = 0 \bmod 8 = 0$ $\quad O(i) \to O(j)$

$j = 12 \bmod 2^m = 12 \bmod 8 = 4$ $\quad 12(i) \to 4(j)$

12 = 0 1 | 1 0 0 (5 biți; 3 biți = m → 4)

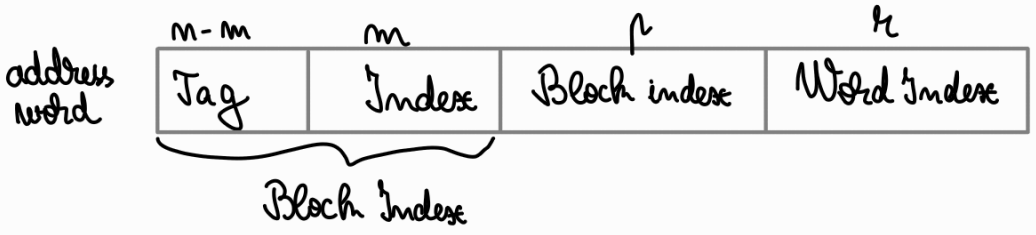
tag



tag length = $n - m$

tag → 01 $\quad$ 100 = adresa = 12

Index



(mai sus sunt inverse, dar sunt doar niște notații r-k)

1 Bloch = 2^r words

1 word = 2^k bytes

Example 1

Se consideră o memorie principală a cărei dimensiune e de 2^{24} cuvinte. Un cuvânt = 1 Byte ($p=0$), un Bloc e 4 cuvinte.

Cum implementăm mapabrea directă a adresei având o memorie cache de 1 kBlock?

La adresa 8192 avem o instrucțiune care are un cod ipotetic 230, la adresa 4102 codul ~~215~~ 71 și la adresa 12 292 codul 170. Unde se mapează în cache?

Date

Size $MM_{M_2} = 2^{24}$ words → nr de biți din address word

1 Block = 4 words

1 word = 1 Byte

Size (M_1/cache) = 1 kBlock = 2^{10}

$$\Rightarrow \frac{2^{24} \text{ words}}{2^2 \text{ word/block}} = 2^{22} \text{ blocks}$$

Address M_2

Address					Block index
0	0	1	2	3	0
4	4	5	6	7	1
8	8	9	10	11	2
⋮	≈	≈	≈		⋮
4092	4092	4093	4094	4095	1023
4096	4096				1024
4100	4100	4101	4102 ~~215~~ 71	4103	1025
⋮	≈	≈	≈		⋮
8188					2044
8192	8192 230	8193	8194	8195	2048
8196					2049
⋮	≈	≈	≈		⋮
12 288					3072
12292	12292 170	12 293	12 294	12 295	3073
12296					3074
$2^{24}-4$					$2^{22}-1$

Cache M1

	V <1>	tag <12>	Data (4 words) <32> 00	01	10	11	
0	1	2	230	d	d	d	8192
1	1 ✓	~~1~~ 3	170	d	~~71~~	d	4102 12292
2							
i							
1023							

0	4096	8192	12288
4	4100	8196	12292
⋮	⋮	⋮	⋮
4092	8188	12284	16380

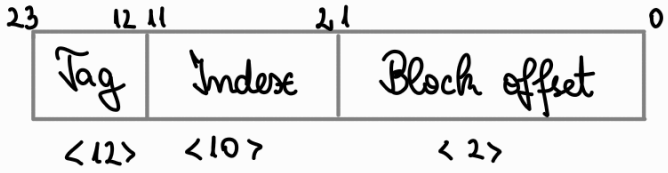
Cum arată adresa cuv. din problemă?

- Adresa va fi pe 24 de biți (size MM = 2^{24})
- Avem 1 word = 1 byte = 2^0 => Word Offset = 0 ($p=0$)
- cache size = 1k blocks = 2^{10} blocks → index = 10

1 block = 2^2 words = $2^2 \cdot 1$ bytes = $2^2 \cdot 2^3$ bits = 2^5 bits = 32 bits

block offset = 2 (2)

24 - 10 - 2 = 12 tag



8192 | 0 0 0 0 0 0 0 0 0 0 0 1 0 | 0 0 0 0 0 0 0 0 0 0 | 0 0 | 2|0|0

4100 | 0 0 0 0 0 0 0 0 0 0 0 1 | 0 0 0 0 0 0 0 0 0 1 | 1 0 | 1|1|2

↳ se mapează la 1

12292 | 0 0 0 0 0 0 0 0 0 0 1 1 | 0 0 0 0 0 0 0 0 0 1 | 0 0 | 3|1|0

↳ se map la 1 → șterge ce e deja

=> tag=3 și pune val

Valid la început e pe 0 → accesezi e pe 1

→ cauți la index, pui valid pe 1, vezi în al câtălea block scrii, ai grijă la tag